Selecting Reliable LLM Math Solutions with Compact Pairwise Ensembles

DASE7506 Advanced Machine Learning Methods | MP2 | Clean-room submission

94.4862%	84.5824%	1.08 s	1.304 MiB
validation	test, post-freeze	test + scoring	inference assets

Abstract

This project selects one of four LLM-generated solutions for each grade-school math problem. I study whether supplied whole-trace embeddings, auditable solution descriptors and within-question comparison provide complementary evidence. The final method combines a pairwise linear ranker with a five-seed ensemble of small pairwise neural rankers, then pools candidates that share a parsed final answer. It uses only the official training split and fixed representations. On validation it achieves 754/798 (94.4862%), compared with 688/798 (86.2155%) for answer-frequency voting and 749/798 (93.8596%) for the linear ensemble alone. The single post-freeze test evaluation is 395/467 (84.5824%). The apparently large validation-test gap is mainly due to split composition: 443 validation questions are unanimous, while every test question has at least two different parsed answers. On disagreement-only validation the final method reaches 311/355 (87.6056%), 3.02 points above test. Complete test prediction and scoring take 1.08 seconds on a four-thread Xeon Platinum 8458P; the NumPy-only inference bundle is 1.304 MiB.

Frozen protocol

Train labels only; validation for development; one test evaluation after source and asset hashes were frozen. NumPy-only offline inference. Four CPU threads. No external data, pretrained weights, answer lookup or evaluation-time API.

1. Research question and hypothesis

The central question is: does a candidate become easier to judge when its reasoning representation is evaluated jointly with explicit, interpretable properties and the other candidates for the same question?

I expected three effects. First, pairwise supervision should better match the selection task than independent correctness classification. Second, simple features such as final-answer support, trace length, number reuse and arithmetic consistency should complement opaque 1,024-dimensional trace embeddings. Third, a small nonlinear ranker should recover interactions missed by a linear model, but only if seed variation is controlled. The experiments also test lexical hashing, fixed random nonlinear features and gradient-boosted trees as alternative explanations for any gain.

2. Method

2.1 Targets and pairwise training

The official parser maps every candidate and reference to a normalized numeric answer. A candidate is positive when its parsed answer equals the reference. For every training question with disagreement, I construct all valid positive-negative candidate pairs. Pair weights are normalized within each question and adjusted for repeated answer support, so questions with many equivalent candidate pairs do not dominate. Both learned branches minimize a pairwise logistic loss on these pairs.

2.2 Auditable descriptors

Each candidate receives 52 deterministic descriptors computed only from the question, four candidate traces and supplied embeddings. They cover:

- trace length, line count, numeric-token counts and formatting markers;
- final-answer validity, magnitude and support among the four candidates;
- overlap with question words and numbers;
- embedding distance and cosine similarity to the within-question mean;
- counts of arithmetic symbols and reasoning cues; and
- a restricted arithmetic audit that safely evaluates short numeric equality

fragments and records inconsistent residuals.

The arithmetic evaluator accepts only numeric constants and `+`, `-`, `*` and `/` AST nodes. Candidate text is never executed.

2.3 Linear branch

The compact baseline learns separate pairwise logistic rankers over raw trace embeddings and the 52 descriptors. Candidate logits are standardized using training-score scales. Validation selects weights 2/3 for raw embeddings and 1/3 for descriptors. This branch alone uses 10.4 KiB and reaches 93.8596%.

2.4 Nonlinear branch and stability control

The neural input concatenates the standardized candidate embedding, its within-question centered embedding and the standardized descriptors (2,100 features total). A single hidden layer has 64 tanh units and a scalar output. An internal stratified 80/20 split of disagreement training questions selects 100 epochs without using validation. I then train five members on all eligible training pairs with seeds 7550-7554. Their individual validation accuracies are 93.23%-93.73% (standard deviation 0.17 points); averaging reaches 93.7343%.

2.5 Frozen aggregation rule

The final candidate score is

`0.55 * linear_score + 0.45 * mean(neural_member_scores)`.

Candidates with the same parsed final answer form a group. A group's value is the mean score of its members plus `0.1 * log(group_size)`. The method chooses the first candidate in the highest-scoring answer group. This choice is local to each question, ignores IDs and works for arbitrary batch sizes. Neural weights are stored in float16 and converted to float32 at startup; validation decisions are unchanged.

3. Experimental protocol

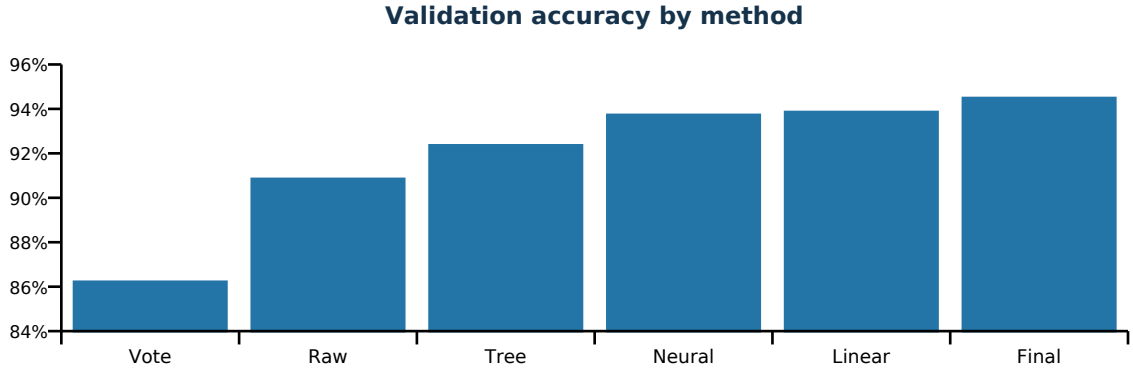
Only the official 4,755-question training split supplies labels. The 798-question validation split is used for architecture and aggregation selection. Source and asset SHA-256 hashes were frozen before any test-reference access. The 467-question test split was then evaluated once; its score did not trigger further tuning. No external data, extra pretrained weights,

answer lookup, evaluation-time LLM or API is used.

All server experiments explicitly set `CUDA_VISIBLE_DEVICES=''` and cap BLAS, OpenMP and related libraries at four threads. An NVIDIA H800 was installed on the machine but was not used; training and evaluation ran on CPU. PyTorch, scikit-learn and CatBoost are training-only dependencies. Frozen inference requires NumPy.

4. Results and ablations

Method	Validation accuracy	Main interpretation
Answer-frequency voting	86.2155%	strong easy-split baseline
Best raw-embedding linear model	90.8521%	embeddings carry correctness signal
Hashed lexical ranker	84.4612%	surface wording alone is weak
Fixed random nonlinear features	89.5990%	nonlinearity without learned projection is insufficient
Best CPU tree ranker	92.3559%	useful but below compact learned branches
Neural, embeddings only	92.2306%	learned interactions help, descriptors still matter
Neural plus descriptors, one seed	93.7343%	explicit evidence adds 1.50 points
Linear embedding plus descriptors	93.8596%	strongest compact single branch
Frozen linear plus five-seed neural	94.4862%	complementary errors add 0.63 points



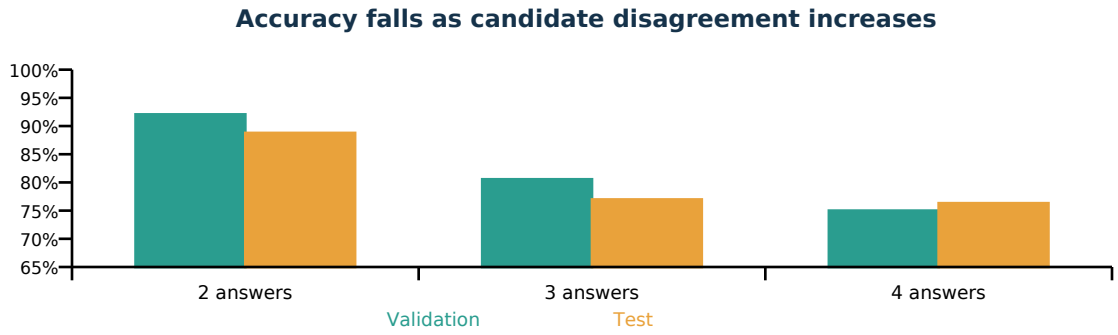
Lexical features do not improve the linear ensemble when blended, so they are rejected. A fixed-random-feature blend reaches 94.1103%, below the final rule. Adding a tree component to the final validation mixture produces one additional correct answer (94.6115%), but costs a more complicated training and inference path. Because that one-question gain is within sampling noise and was found after a broader auxiliary search, I reject it. This is a deliberate accuracy-complexity trade-off rather than reporting the largest observed validation number.

The linear baseline answers 749 validation questions correctly; the final rule answers 754. Across five neural seeds, mean single-model accuracy is 93.4336% with a 0.1700-point standard deviation. The seed average and a broad plateau of linear/neural mixture weights give the same 754 correct decisions, providing more evidence than a single lucky initialization.

5. Difficulty and error analysis

The overall validation score is not directly comparable with test because 443 of 798 validation questions have only one distinct valid parsed answer among the candidates. All 443 are necessarily easy for answer-aware aggregation. On the 355 validation questions with two or more answers, the majority baseline is 69.0141% and the final method is 87.6056%. Every test question is in this harder regime; test accuracy is 84.5824%.

Distinct parsed answers	Validation final	Test final
2	209/227 (92.07%)	269/303 (88.78%)
3	87/108 (80.56%)	97/126 (76.98%)
4	15/20 (75.00%)	29/38 (76.32%)



Performance falls as disagreement increases. The sharper failure mode is the number of correct candidates. On test, accuracy is 66.89% when only one of four candidates is correct, 88.28% when two are correct and 97.08% when three are correct. Thus the method is effective at exploiting redundant correct evidence but still struggles to recognize a lone correct trace surrounded by plausible wrong solutions.

Against answer-frequency voting on test, the final method fixes 102 majority errors but introduces 30 new errors, for a net gain of 72 questions (15.42 points). This confirms that learned reasoning evidence matters, while the 30 regressions show that the support prior can still override a uniquely correct minority answer.

6. Resource use

The frozen asset directory is 1,366,950 bytes (1.304 MiB), well below the 16 MiB limit. On an Intel Xeon Platinum 8458P using four threads, complete prediction plus independent scoring takes 1.61 seconds for validation and 1.08 seconds for test, versus the 30-second limit. Test prediction alone reports 0.834 seconds. The five neural members are fused into one array bundle; float16 storage halves their disk footprint without changing validation choices.

Measured exploratory training scripts total about 356 seconds: 20.8 seconds for linear search, 14.5 lexical, 13.6 fixed random features, 236.6 trees, 36.4 neural architecture search and 34.1 seed stability. This reinforces the final trade-off: trees consumed most experiment time but were not selected.

7. Limitations

The validation set is small after removing unanimous questions, so differences of one to five questions are uncertain. Validation also influenced multiple architecture and mixture comparisons; reporting every maximum as a real gain would overstate evidence. The final choice therefore prioritizes a broad weight plateau, repeated seeds and a simple NumPy runtime.

The arithmetic audit is intentionally conservative. It misses prose-level reasoning errors, units, time interpretation and equations containing variables or complex LaTeX. Whole-trace embeddings remain opaque, and this experiment cannot prove which semantic properties they encode. Future work should use training-only cross-validation to estimate aggregation weights and explicitly train a gate for lone-correct-candidate cases, without consulting test.

8. Reproducibility and disclosure

The repository includes all training scripts, frozen assets, hashes, raw experiment JSON, prediction files, official score JSON, contract tests and exact commands. Contract checks confirm batch independence, question-order independence, candidate-permutation equivariance, support for unseen IDs and an empty bank. OpenAI Codex provided substantive help with design, implementation, experimentation, server operation, analysis and documentation. This assistance is disclosed in `AI_DISCLOSURE.md`; responsibility for understanding and presenting the work remains with the submitting student.

References

1. DASE7506 MP2, *Selecting Reliable LLM Reasoning*, official project guide, 2026.
https://github.com/xudongwu-0/DASE7506-MP2

2. Karl Cobbe et al., *Training Verifiers to Solve Math Word Problems*, 2021.

3. Liudmila Prokhorenkova et al., *CatBoost: unbiased boosting with categorical features*, NeurIPS 2018.